Birla Institute of Technology and Science, Pilani
Hyderabad Campus
Semester I, Academic Year 2026–27 | CS F425 – Deep Learning

Project Assignment - I

Total: 30 Marks | Course Weight: 10%

Instructor In-Charge	Prof. Sk Aziz Ali | skaziz.ali@hyderabad.bits-pilani.ac.in
Assignment Start	20/09/2026 at 11:59 PM
Submission Deadline	**16/10/2026 at 05:00 PM – NO EXTENSION**

MARK DISTRIBUTION: Classification 12 Marks + Instance Segmentation 15 Marks + Submission & Organization 3 Marks = 30 Marks

1. General Instructions

- Work in your pre-assigned group. Only one submission per group is permitted.
- Create a **PRIVATE (NOT public)** GitHub repo named `CSF425_2026_XX`, where XX is your assigned group ID.
- The repository must contain `src/`, `README.md`, `config.yml` (or `environment.yml`), `results/`, `checkpoints/`, and `scripts/`.
- Only **ONE** group member must email the final submission. Subject: `[CSF425_2026_XX_Assignment-I]`. Attach exactly one archive: `CSF425_2026_XX_Assignment-I.zip`.
- Use the supplied HPC account for large datasets/checkpoints. Kaggle Notebooks may be used as an alternative GPU environment; accelerator availability/quota can change.
- Do not commit private SSH keys, dataset archives, or large checkpoints to GitHub.

2. HPC Access and SSH-Key Security

The private SSH key is confidential. Never email it, upload it to GitHub, share it with another group, or make it world-readable.

1. With you, a public and private keys to login to BITS Pilani HPC will be shared (see https://sharanga.hpc.bits-hyderabad.ac.in/ and job scheduling process). Save the private key as `csisdl_XX` and set strict permissions:
   ```
   chmod 600 csisdl_XX
   ```
2. Recommended explicit-key login:
   ```
   ssh -X -i ./csisdl_XX csisdl_XX@hpc.bits-hyderabad.ac.in
   ```
3. If the key is already registered in SSH config/agent:
   ```
   ssh -X csisdl_XX@hpc.bits-hyderabad.ac.in
   ```
4. Optional alias in `~/.bashrc`:
   ```
   alias loginhpc='ssh -X -i /absolute/path/csisdl_XX csisdl_XX@hpc.bits-hyderabad.ac.in'
   ```

Use only your assigned account. GPU access is shared; terminate idle jobs/processes and checkpoint long experiments.

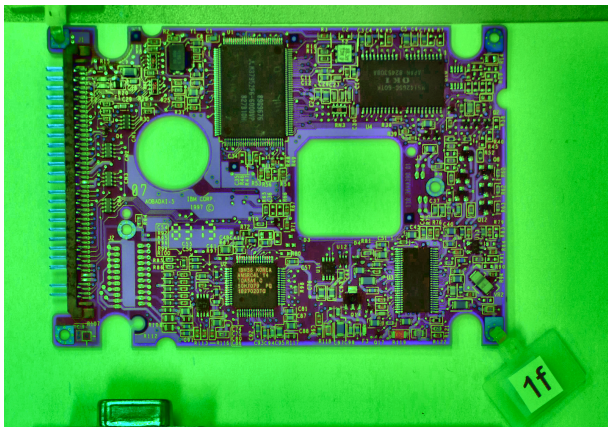
3. Dataset and Problem Definition

Students must solve two tasks on a common 25-class taxonomy: (i) component classification and (ii) instance segmentation. The assignment rewards predictive quality and parameter efficiency rather than model scale.

Dataset	Train	Test	Role	Notes
Graph-W	37	10	Classification + segmentation source	Derived from the WACV 2019 PCB dataset.
Graph-F	115	47	Classification + segmentation source	GraphPCB variant derived from FPIC/FPIC-component.
WACV 2019 PCB	–	–	Background/source data	47 high-resolution images, 31 component types, approximately 62k component instances.
FPIC-component	–	–	Canonical label taxonomy	Use the 25 target classes below.

Canonical 25-class rule. All experiments, plots, JSON files, color overlays, and reported metrics must use exactly the 25 target labels below. Extra source labels must be mapped to this taxonomy or marked ignore/void. Document the mapping in `README.md` and `data_mapping.json`.

ID	Class	Color	ID	Class	Color
1	Resistor (R)	#E53935	14	Motor (M)	#039BE5
2	Capacitor (C)	#1E88E5	15	Button (BTN)	#757575
3	Integrated Circuit (U)	#00ACC1	16	Ferrite Bead (FB)	#FDD835
4	Discrete Transistor (Q)	#8E24AA	17	CRA	#212121
5	Connector (J)	#FB8C00	18	Switch (SW)	#7E57C2
6	Inductor (L)	#43A047	19	Transformer (T)	#F4511E
7	Resistor Coil (RA)	#6D4C41	20	Fuse (F)	#2E7D32
8	Diode (D)	#D81B60	21	Vacuum Tube (V)	#283593
9	Resistor Network (RN)	#3949AB	22	Light Emitting Diode (LED)	#7CB342
10	Test Point (TP)	#C0CA33	23	Switch (S)	#546E7A
11	Integrated Circuit (IC)	#00897B	24	QA	#C2185B
12	Plug (P)	#FFB300	25	Jumper Link (JP)	#F9A825
13	Thyristor (CR)	#5E35B1			



Example PCB image



Example colored component-mask visualization

4. Exercise 1 – PCB Component Classification [12 Marks]

Objective. Implement and train your own deep neural network (can be pretrained and later finetuned) to classify PCB component instances into the fixed 25 classes. This is a component-classification task. For full-board visualization, use annotated/derived component boxes and color each box by the predicted class.

- Use official Graph-W and Graph-F train/test partitions. Never move test samples into training.
- Report Precision (P_{macro}), Recall (R_{macro}), and F1 ($F_{1,macro}$) plus per-class values. Accuracy alone is insufficient for this imbalanced problem.
- Plot $P(t)$, $R(t)$, and $F_1(t)$ for thresholds $t \in [0, 1]$. Select t^* using validation data and freeze it before test evaluation.
- Run at least three distinct model/configuration trials. Log total and trainable parameter counts for every trial.
- Pretrained weights are allowed, but architecture/source and trainable parameter counts must be disclosed.

Measure the classification performance index with: $S_{cls} = 0.20P_{macro} + 0.20R_{macro} + 0.60F_{1,macro}$.

5. Exercise 2 – PCB Component Instance Segmentation [15 Marks]

Objective. Train an instance-segmentation model that outputs a separate pixel mask and 25-class label for each component instance. Box-only detection is not a substitute. Use the exact same 25-class taxonomy and color mapping as Exercise 1. Overlay translucent predicted masks on test images.

- Report AP50, $mAP@[0.50 : 0.95]$, mask F1, class-wise AP, and class-wise F1.
- Plot precision-recall behavior and $F_1(t)$ for confidence thresholds $t \in [0, 1]$; include AP/mAP statistics as defined by the selected framework.
- Run at least three model/configuration trials; report parameters, loss terms, validation metrics, and test results.
- Large pretrained segmenters are allowed, but no grading advantage is awarded merely for scale.

Measure the Segmentation performance index with: $S_{seg} = 0.20AP_{50} + 0.50mAP_{50:95} + 0.30F_{1,mask}$.

6. Efficiency-Aware Relative Grading of Your Submission

It is decided to grade your submission relative to all submissions, which will depend upon several factors. Let P_i be the number of trainable parameters in the best model for group i, and let P_{med} be the median parameter size of the DL models across all valid submissions for that task. Then the efficiency E_i of your DL model will be defined by:

$$E_i = \text{clip}\left(1 + 0.05[\log_{10}(P_{med}) - \log_{10}(P_i)],\ 0.90,\ 1.10\right).$$

Now you may feel you want to crowd-source high-definition images of the PCB component and fill those classes with extra image-label pairs. This is a valid step, and you are allowed to do so. However, you will be penalized beyond the **1.5x** size of the original data, where Let $r_i = N_{\text{distinct train data used}}/N_{\text{official train}}$. On-the-fly transforms do not increase r_i. Then,

$$A_i = \begin{cases} 1, & r_i \le 1.5 \\ 1.5/r_i, & r_i > 1.5 \end{cases} \qquad U_{cls,i} = S_{cls,i} E_i A_i, \quad U_{seg,i} = S_{seg,i} E_i A_i.$$

The Relative core marks or grading will be done based on:

$$M^{core}_{cls,i} = 8\frac{U_{cls,i}}{\max_j U_{cls,j}}, \qquad M^{core}_{seg,i} = 10\frac{U_{seg,i}}{\max_j U_{seg,j}}.$$

Test labels must never be used for model selection, threshold tuning, pseudo-label acceptance, or manual cleanup. Any leakage invalidates the relative score.

Component	Classification	Segmentation	Assessed
Relative performance + efficiency	8.0	10.0	Utility formula; normalized against best valid group.
Methodology + ablation	2.0	2.5	Architecture rationale, loss, imbalance strategy, ablations, threshold selection.
Required outputs + analysis	2.0	2.5	Plots, overlays, JSON logs, checkpoint, inference, reproducibility.
TOTAL	**12.0**	**15.0**	

7. Mandatory Outputs Necessary for EACH Exercise as part of Submission

Submissions	Required content	Mandatory naming
Source code	Complete training/evaluation source under `src/classification/` and `src/segmentation/`.	–
Test visualizations	Classification: colored predicted-class boxes. Segmentation: translucent predicted instance masks using the same palette.	`results/classification/*.png`; `results/segmentation/*.png`
Plots	Classification: $P(t), R(t), F_1(t)$, train/val loss, accuracy where meaningful, confusion matrix. Segmentation: AP/mAP/PR/F1, losses, class-wise AP/F1.	Same results folders
JSON statistics	Same key schema across trials: `config_name, seed, epochs, total_params, trainable_params, train_loss, val_loss, val_metrics, test_metrics, threshold, data_ratio, runtime, checkpoint_name`.	`classification_stats.json`; `segmentation_stats.json`
Best checkpoints	Best evaluated checkpoints.	`best_classification.ckpt`; `best_segment.ckpt`
Standalone inference Code	Each shell script must invoke one standalone Python file that contains all project-specific construction, forward, pre/post-processing, and metrics/visualization; only external packages may be imported.	`run_classification.sh`; `run_segmentation.sh`; `inference_classification.py`; `inference_segmentation.py`
README	Step-by-step environment, data mapping, training, evaluation, inference, paths, checkpoints, and reproduction instructions.	`README.md`

Recommended repository structure (Email ONE .zip File)

```
CSF425_2026_XX/  README.md, config.yml, data_mapping.json, src/, scripts/, checkpoints/, results/
```

8. Submission & Organization [3 Marks – All-or-Nothing]

All 3 marks are awarded only if every mandatory item below is correct. Otherwise this section receives 0/3; marks earned in Exercises 1 and 2 are unaffected.

☐ Private GitHub repository uses the exact group naming convention and required folder structure.

☐ Final archive is exactly `CSF425_2026_XX_Assignment-I.zip`; no private SSH key or dataset dump is included.

☐ One email only, subject exactly `[CSF425_2026_XX_Assignment-I]`.

☐ Both required checkpoints exist with exact filenames and are readable.

☐ Both inference shell scripts execute from a clean environment with the documented commands.

☐ Plots/overlays use the required colors; JSON files are valid and consistent across ablations.

☐ README contains complete reproduction instructions, software/hardware versions, and parameter counts.

☐ External data, pretrained weights, and adapted third-party code are clearly cited.

9. Experimental Integrity and Reproducibility Rules

- Set/report random seeds. Report mean ± standard deviation when multiple seeds are run.
- Standard external libraries and pretrained weights are allowed; an off-the-shelf pipeline without meaningful training/fine-tuning and analysis is insufficient.
- Do not modify test annotations, hand-correct predictions, or cherry-pick quantitative reporting. Metrics must use the entire prescribed test split.
- For external online images, record source URL, license/usage status where available, original label, mapped class, and train-only membership. No duplicates of test images.
- The Instructor may rerun inference on hidden/sample data using the submitted checkpoint and scripts.

10. Dataset References

- GraphPCB: https://github.com/BlueSkyLT/GraphPCB
- FPIC-component: https://datasetninja.com/fpic-component
- PcbSegClassNet paper: https://www.researchgate.net/publication/364257730_Pcbsegclassnet_-_a_Light-Weight_Network_for_Segmentation_and_Classification_of_Pcb_Component
- Kaggle Notebooks: https://www.kaggle.com/docs/notebooks
- Kaggle GPU usage: https://www.kaggle.com/docs/efficient-gpu-usage